PRÉSENTATION DU PROJET / 01

# StudyTasks

Une application Python de gestion de tâches, présentée dans une interface web locale avec Streamlit.

## Objectif et utilisation

StudyTasks aide à organiser le travail quotidien : créer des tâches, fixer leurs priorités, les classer par catégories, suivre leur état et conserver les données entre deux sessions. Le projet réunit la gestion de listes et dictionnaires, les fonctions, les événements d'interface et la manipulation de fichiers.

## Fonctionnalités réalisées

| Fonction | Comportement dans l'application |
|---|---|
| Créer et modifier | Saisie d'un titre, d'une priorité, d'une catégorie et d'une échéance facultative dans une fenêtre de dialogue. |
| Terminer ou réactiver | Passage entre les états Pending et Completed ; l'action peut être annulée en repassant la tâche en attente. |
| Supprimer | Suppression de la tâche sélectionnée après confirmation. |
| Organiser | Recherche par titre/catégorie, filtres par état et catégorie, tri par priorité, date ou ordre d'ajout. |
| Conserver les données | Sauvegarde automatique en JSON après une modification acceptée et rechargement au démarrage. |
| Exporter | Téléchargement CSV de toutes les tâches, même lorsqu'un filtre en masque certaines. |

## Lire les indicateurs

Les compteurs portent sur **toutes les tâches enregistrées**, indépendamment des filtres. Le pourcentage d'une catégorie d'état vaut : **nombre concerné ÷ nombre total × 100**. Avec 4 tâches dont 1 terminée : Completed = 25 % et Pending = 75 %.

### Ce que signifie « Overdue »

Une tâche est en retard si elle n'est pas terminée et si son échéance précède la date du jour. Une tâche due aujourd'hui n'est pas encore en retard. Les tâches en retard font partie des tâches Pending : ces deux compteurs ne sont donc pas indépendants.

# Organisation du programme

Trois responsabilités : l’interface, les règles de gestion et la persistance.

| Fichier | Responsabilité |
| --- | --- |
| app.py | Construit l’interface Streamlit, gère la sélection, les dialogues, les indicateurs et le téléchargement CSV. |
| tasks.py | Valide les détails et contient add_task, update_task, toggle_task, delete_task, filter_tasks et sort_tasks. |
| storage.py | Lit et écrit tasks.json, valide sa structure et détermine le séparateur CSV régional. |
| requirements.txt | Déclare Streamlit et pandas avec les versions autorisées. |
| .streamlit/config.toml | Définit le thème et l’adresse locale du serveur. |
| START.bat | Place le terminal dans le bon dossier, prépare .venv et lance Streamlit sous Windows. |
| test_project.py | Contient 13 tests automatiques des données, opérations et fichiers. |

## Comment une tâche est représentée

En mémoire, les tâches forment une **liste de dictionnaires Python**. Chaque dictionnaire possède les champs ci-dessous. Les noms techniques sont conservés en anglais dans le code et le fichier JSON.

| Champ | Signification |
| --- | --- |
| id | Identifiant numérique permettant de retrouver la bonne tâche, même après un tri ou avec des titres identiques. |
| title / completed | Titre de la tâche / booléen indiquant si elle est terminée. |
| priority / category | High, Medium ou Low / catégorie, par exemple Personal, Health ou Work. |
| due_date | Échéance au format AAAA-MM-JJ ; chaîne vide si aucune échéance n’est définie. |

## Exemple de donnée enregistrée

```
{ "id": 1, "title": "Réviser Python",
  "completed": false, "priority": "High",
  "due_date": "2026-11-10", "category": "Studies" }
```

# Du clic à la sauvegarde

Comprendre le parcours d’une action dans les fichiers du projet.

## 1. Ajouter ou modifier une tâche

Le bouton **New task** ouvre le dialogue **task_editor()**. L’utilisateur renseigne les champs. Pour une échéance, il choisit une année, un mois et un jour ; **calendar.monthrange()** détermine les jours réellement disponibles, y compris en année bissextile.

Lors de l’enregistrement, app.py recharge les données, puis appelle **add_task()** ou **update_task()**. Le titre et la catégorie doivent être non vides ; la priorité et la date sont vérifiées. En cas de réussite, **save_tasks()** écrit le fichier et l’interface est recalculée.

## 2. Terminer, supprimer et retrouver la bonne tâche

**toggle_task()** inverse le booléen completed. **delete_task()** enlève la tâche de la liste après confirmation. Ces opérations utilisent **id**, plutôt que le titre ou une position fixe dans le tableau. L’application conserve aussi l’identifiant sélectionné dans **st.session_state**.

La clé du tableau dépend de son contenu. Lorsqu’un tri, un filtre ou une modification change les lignes affichées, cela évite de réutiliser une ancienne sélection de ligne pour une autre tâche.

## 3. Comprendre les réexécutions Streamlit

Une interaction peut réexécuter le programme afin de reconstruire l’affichage. **st.session_state** conserve des informations d’interface, comme la sélection et les filtres, pendant la session. **tasks.json** conserve les tâches sur le disque. Ce sont deux rôles différents.

## 4. Sauvegarder sans remplacer un fichier par une écriture partielle

**json.load()** transforme le contenu JSON en objets Python. **json.dump()** réalise l’écriture inverse. Les fichiers sont ouverts avec **with** et un encodage UTF-8. La sauvegarde écrit d’abord dans un fichier temporaire, puis remplace le fichier cible lorsque l’écriture a réussi.

Un fichier absent correspond à un premier lancement. Un fichier illisible ou une structure invalide provoque un message d’erreur, sans être écrasé. Une sauvegarde échouée ne valide pas le changement sur le disque.

## 5. Exporter au format CSV

Le bouton d’export de l’interface utilise **csv_bytes()** dans app.py pour préparer le téléchargement. Les colonnes sont ID, Task, Status, Priority, Deadline et Category. Le séparateur peut être régional, une virgule ou un point-virgule ; l’encodage UTF-8 avec indicateur BOM facilite la lecture des accents dans Excel.

Le module storage.py contient également une fonction **export_csv()** pour écrire un export sur disque ; le bouton Streamlit utilise le téléchargement en mémoire, et non cette fonction.

# Présenter et vérifier

Une démonstration courte, puis les points à pouvoir expliquer au professeur.

## Démonstration conseillée : environ 5 minutes

| Durée | Action à montrer |
|---|---|
| 0:00 - 1:20 | Présenter l'objectif et les compteurs ; créer deux tâches avec des priorités, catégories et échéances différentes. |
| 1:20 - 2:30 | Trier et filtrer, terminer une tâche et montrer la mise à jour des pourcentages. |
| 2:30 - 3:40 | Modifier une tâche, exporter en CSV et expliquer le choix du séparateur pour Excel. |
| 3:40 - 5:00 | Fermer puis relancer pour vérifier la persistance ; expliquer les rôles de app.py, tasks.py et storage.py. |

## Questions auxquelles savoir répondre

**Pourquoi utiliser un identifiant ?** Pour agir sur la bonne tâche après un tri ou lorsque plusieurs tâches portent le même titre.

**Pourquoi JSON ?** Pour enregistrer simplement la liste de tâches et leurs champs, puis les reconstruire à la session suivante.

**Pourquoi conserver les dates en AAAA-MM-JJ ?** Parce que les dates ISO complètes de ce format se comparent dans l'ordre chronologique. À l'affichage, l'application utilise JJ/MM/AAAA.

## Vérifications et limites à annoncer

Le projet comprend **13 tests de données et fichiers**. L'interface a aussi été vérifiée avec l'outil AppTest de Streamlit : ajout, modification, validation des dates, filtres et suppression. La démonstration finale doit être testée sur le poste de présentation.

```
.\.venv\Scripts\python.exe -m unittest -v
```

Cette version est destinée à un usage local avec une session d'édition active. Les données sont dans un fichier JSON, sans base de données partagée. Une première installation des dépendances nécessite Internet.

### Écart avec le cahier des charges initial

Le document du professeur mentionne **Tkinter** et une **Listbox**. Cette édition utilise **Streamlit** et un tableau interactif. Les fonctionnalités métier sont présentes, mais ces deux choix techniques diffèrent du sujet initial et doivent être signalés.

Documentation rédigée à partir du ZIP actuel. Les fichiers Python, le lanceur, la configuration et les dépendances n'ont pas été modifiés pour cette traduction.